



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



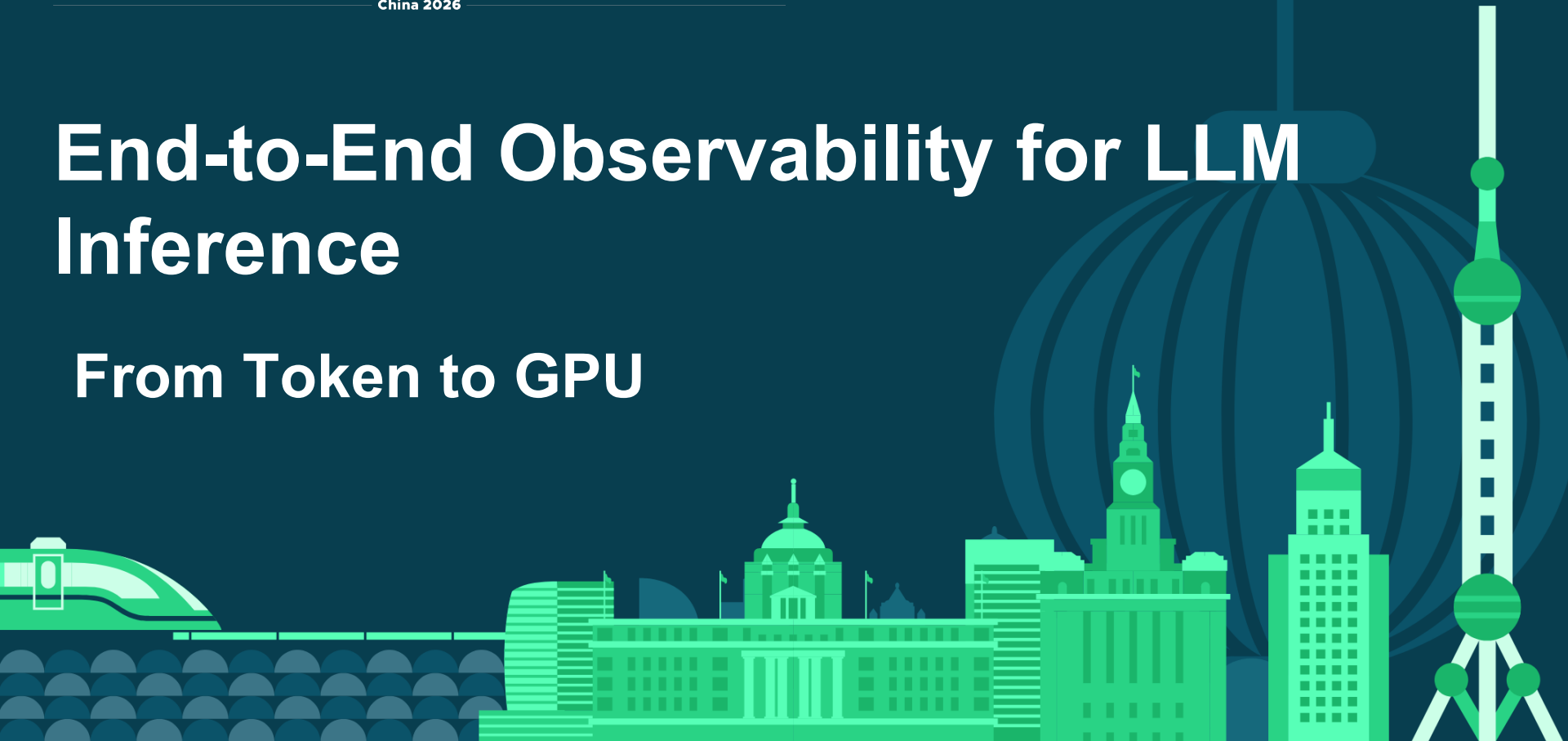
PyTorch
CONFERENCE

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

End-to-End Observability for LLM Inference

From Token to GPU



Co-Speakers



Jared Tan, DaoCloud

OpenTelemetry Helm charts
approver

OpenTelemetry collector contrib
triager

Apache Skywalking PMC
Member



Murphy Chen, DaoCloud
Senior Software Engineer

OpenTelemetry collector
contrib triager



Nicole Li

Inference Acceleration Team
DaoCloud

LLM-D, vLLM, Istio Committer



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

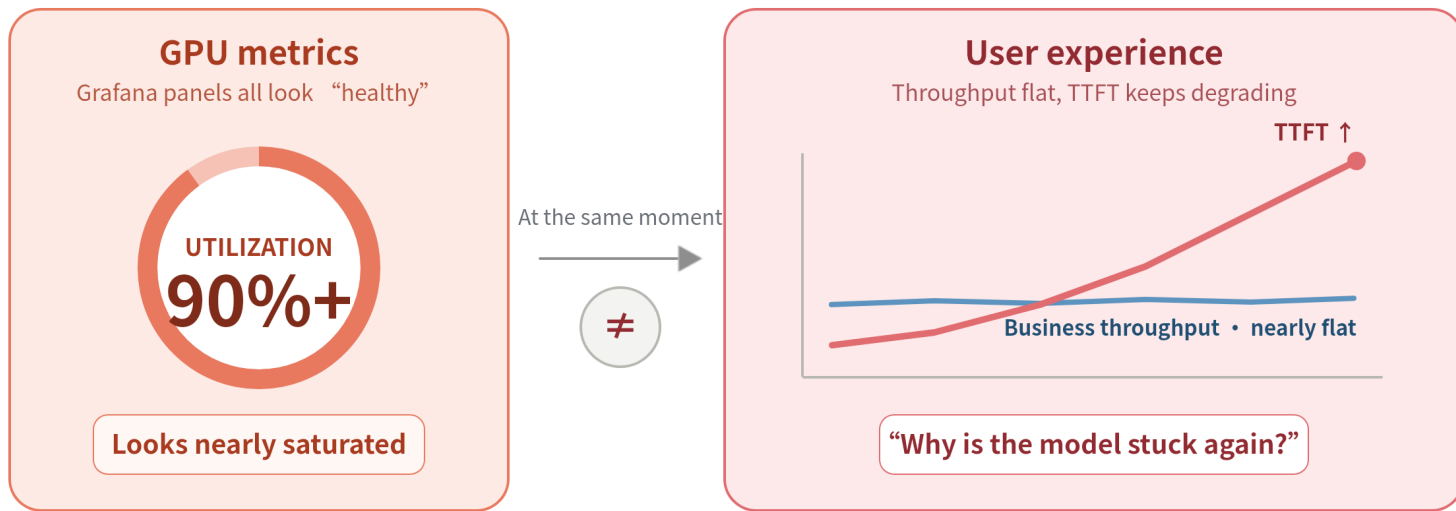
Agenda

1. A counterintuitive observation
2. LLM inference vs. microservices: three differences
3. End-to-end observability: eight layers, Token to GPU
4. Unified conventions and cross-layer correlation
5. Three cases: TTFT spikes / fake-busy GPU / KV Cache
6. Diagnostic paradigm and open-source toolchain
7. Summary



A Counterintuitive Observation

The busier the GPU looks, the worse the user experience

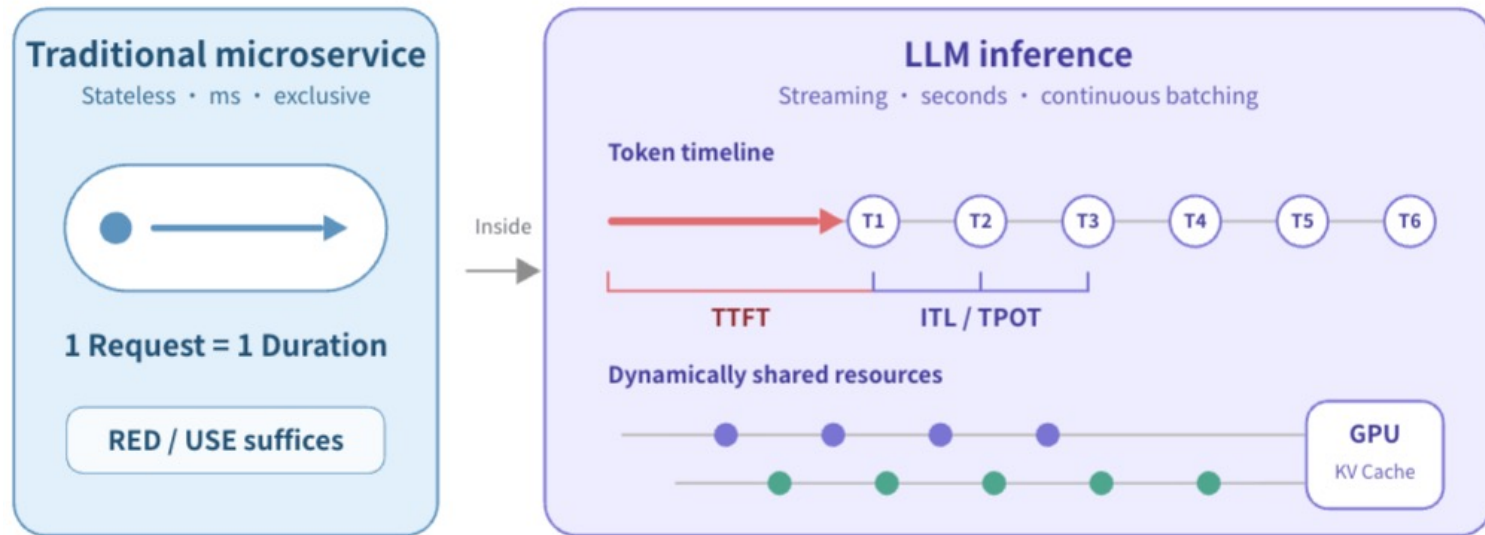


High utilization $\neq$ healthy system

GPU utilization no longer describes the real state; scaling up may not fix the root cause

Traditional Monitoring's Limits: Request vs. Token Thinking

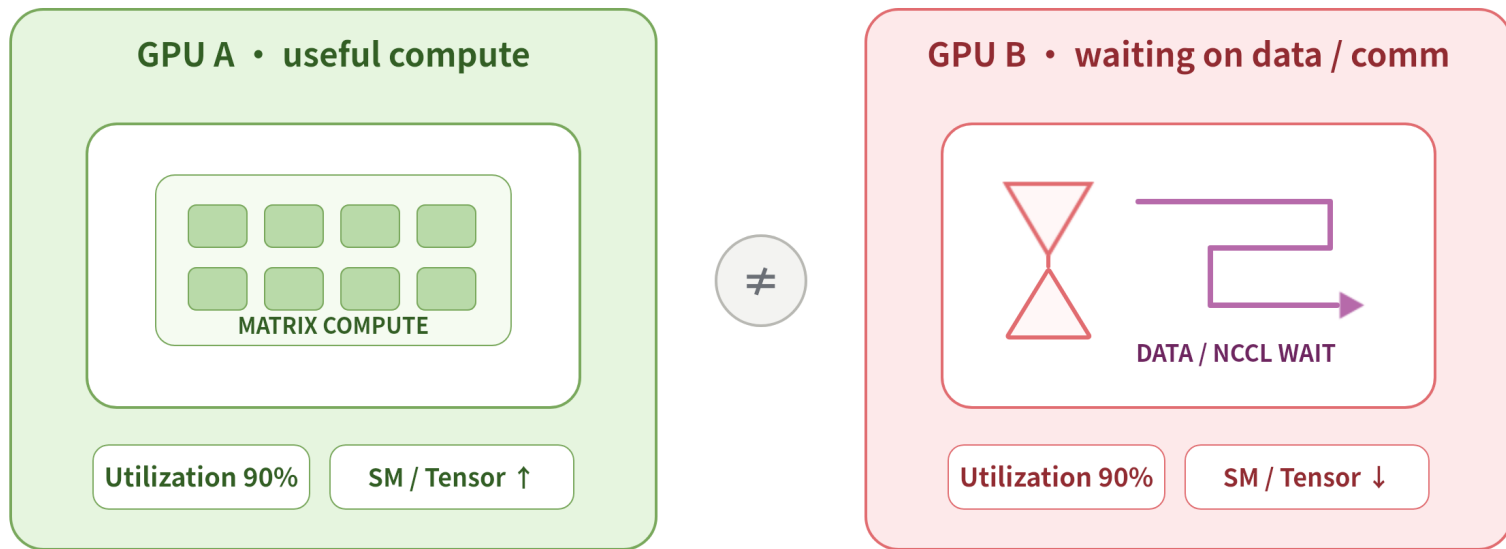
The smallest observable unit moves from a request to inside the request



Latency turns from a point into a curve; resources go from exclusive to shared

The Lie of GPU Utilization

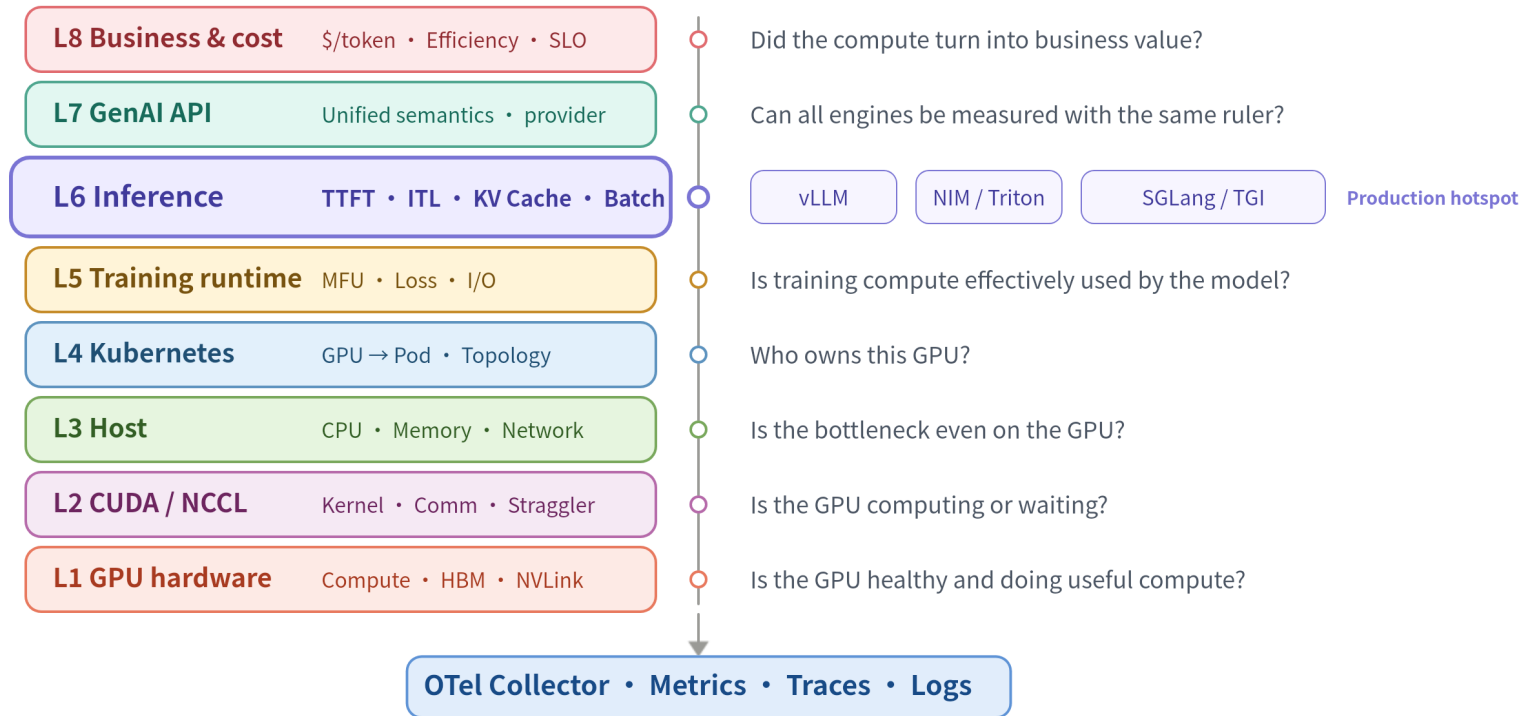
Both GPUs show 90%, yet their health can be completely different



Utilization ↑ + SM Occupancy ↓ + Tensor Core Activity ↓
= waiting, not computing

Layered Observability Architecture Overview

Answers production questions layer by layer • L6 is the hotspot • OTel spans the whole path

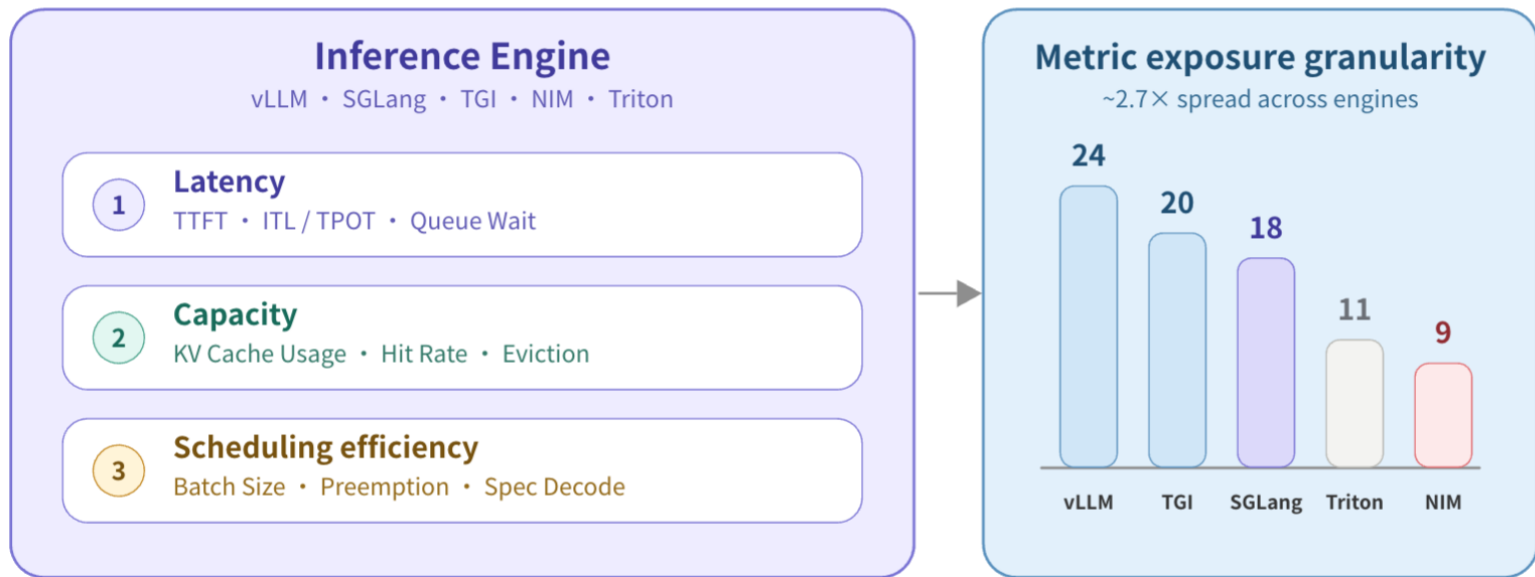


KubeCon

Cloud

The Inference Engine Layer: Where Problems Really Live

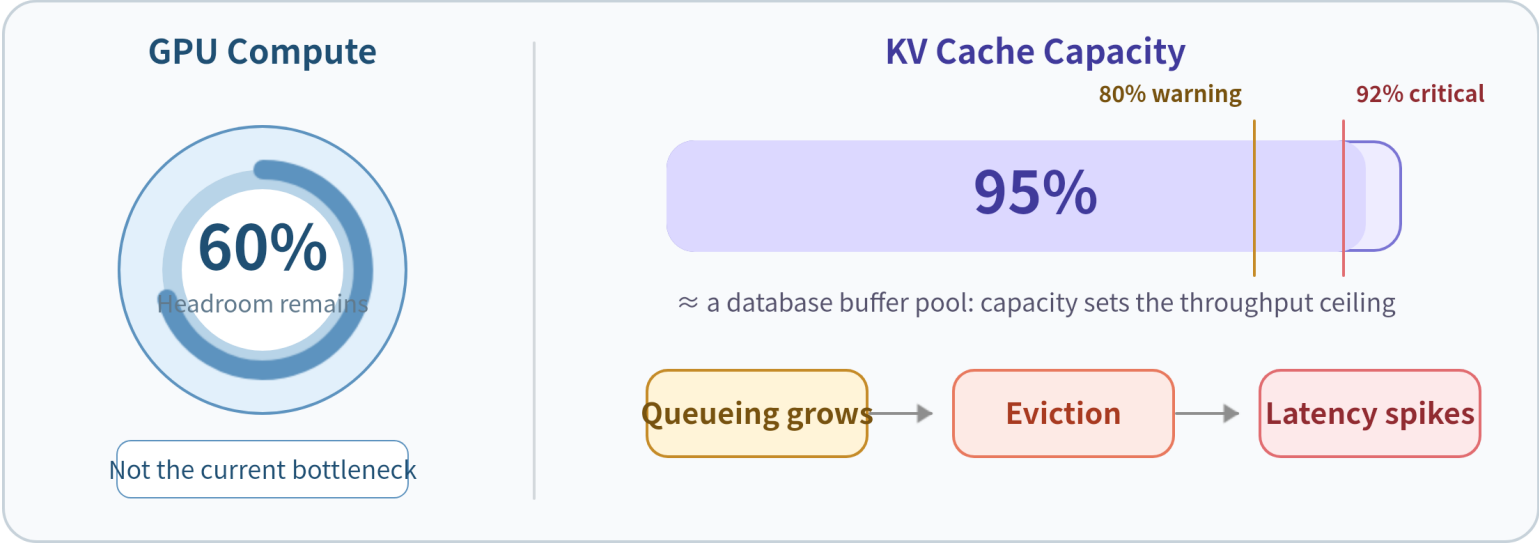
Three metric groups: why slow, enough capacity, efficient scheduling



Different engines, different granularity; the observability model must be unified first

The Key Reframe: KV Cache Matters More Than GPU Utilization

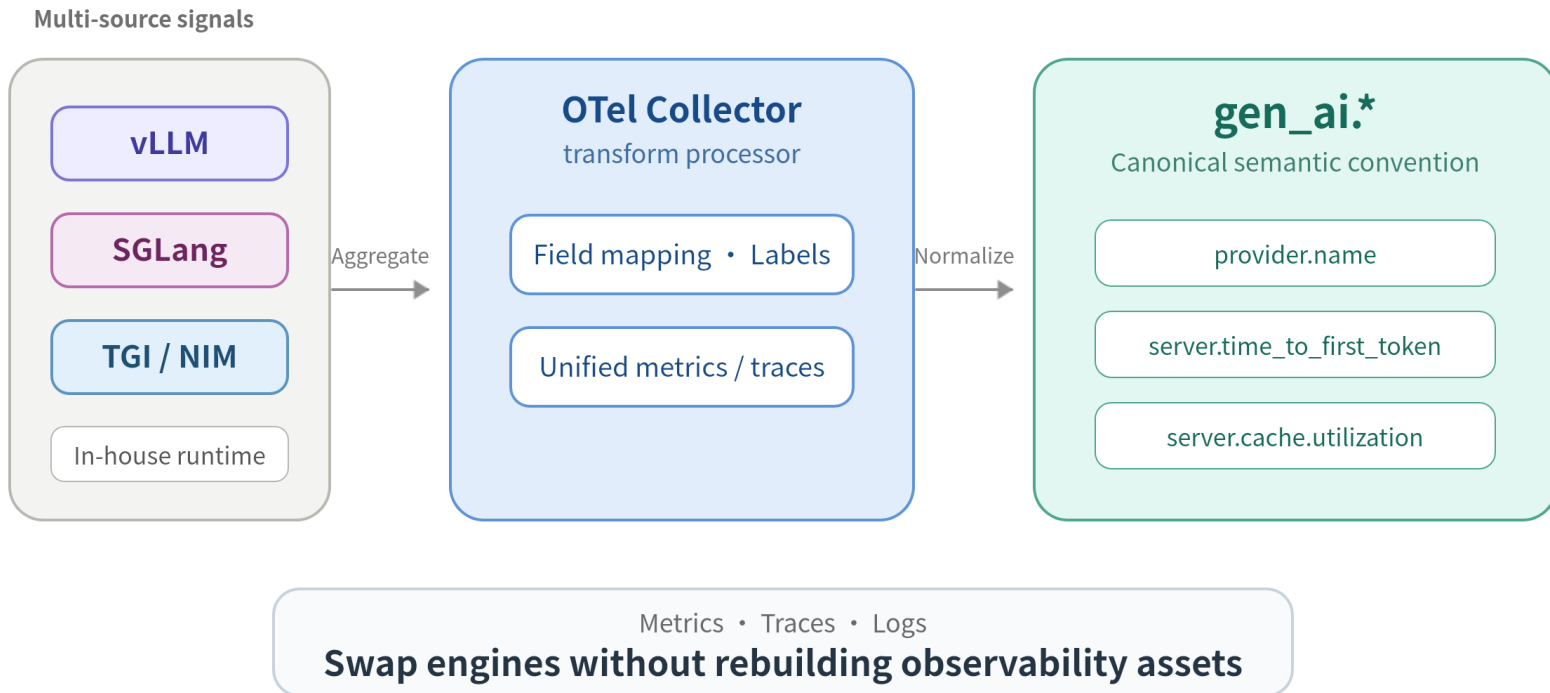
Compute still has headroom, but capacity triggers the chain reaction first



Capacity thresholds must be calibrated to your context-length distribution
The throughput ceiling is not always written in FLOPS

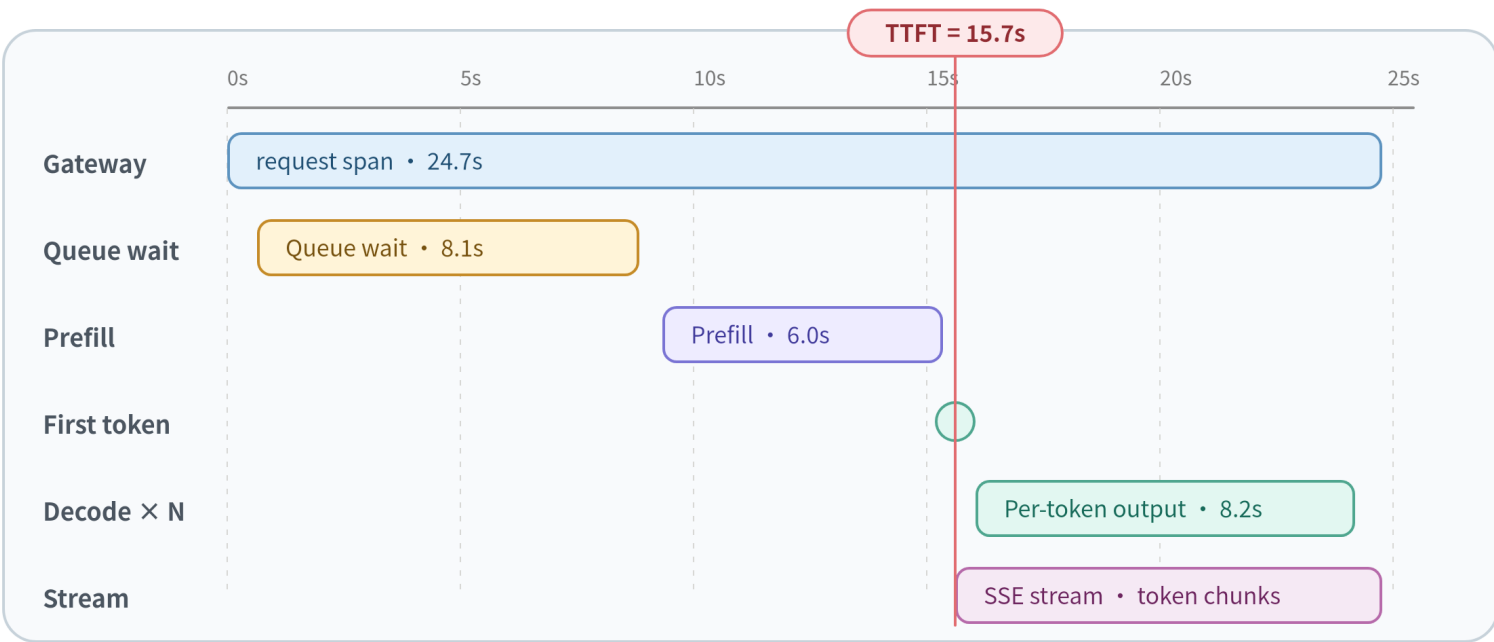
A Common Standard: OpenTelemetry GenAI Semantic Conventions

Many engines, one set of dashboards and alert rules



Cross-Layer Correlation: One Streaming Request's Trace

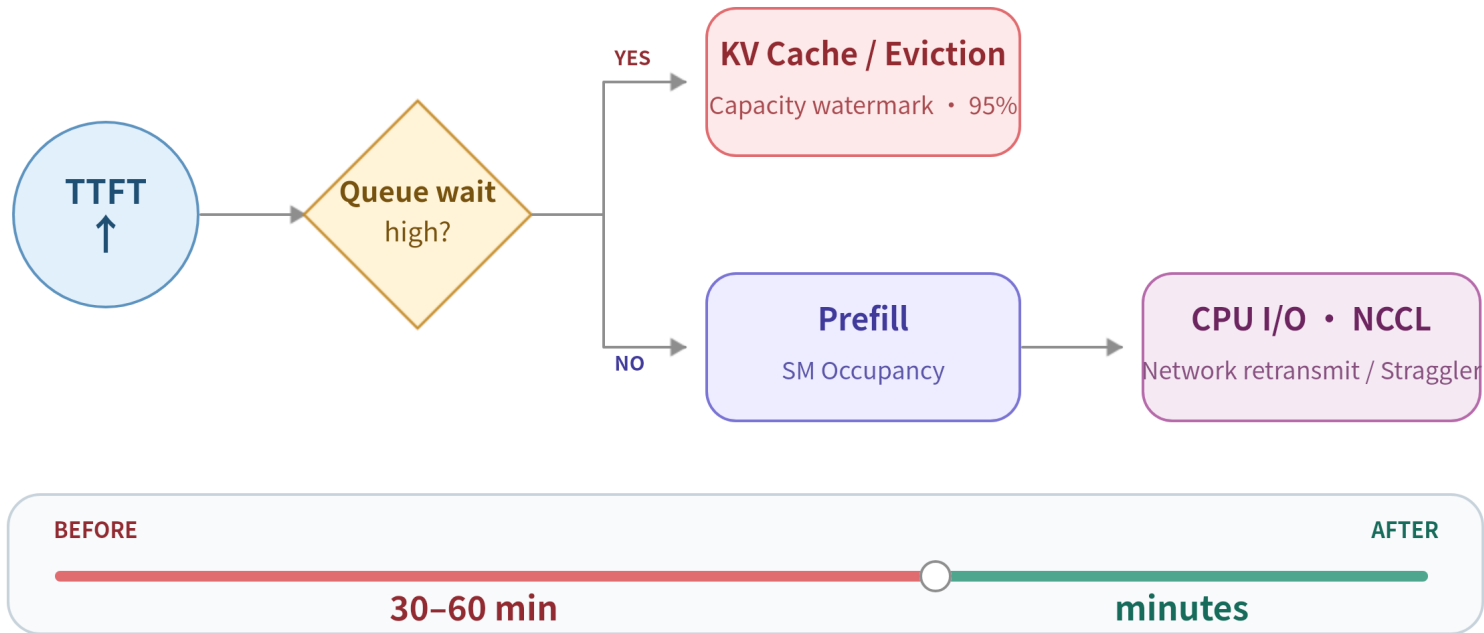
One trace ID across gateway • engine • runtime • GPU execution



Queue → prefill → streaming output: see at a glance which stage is slow

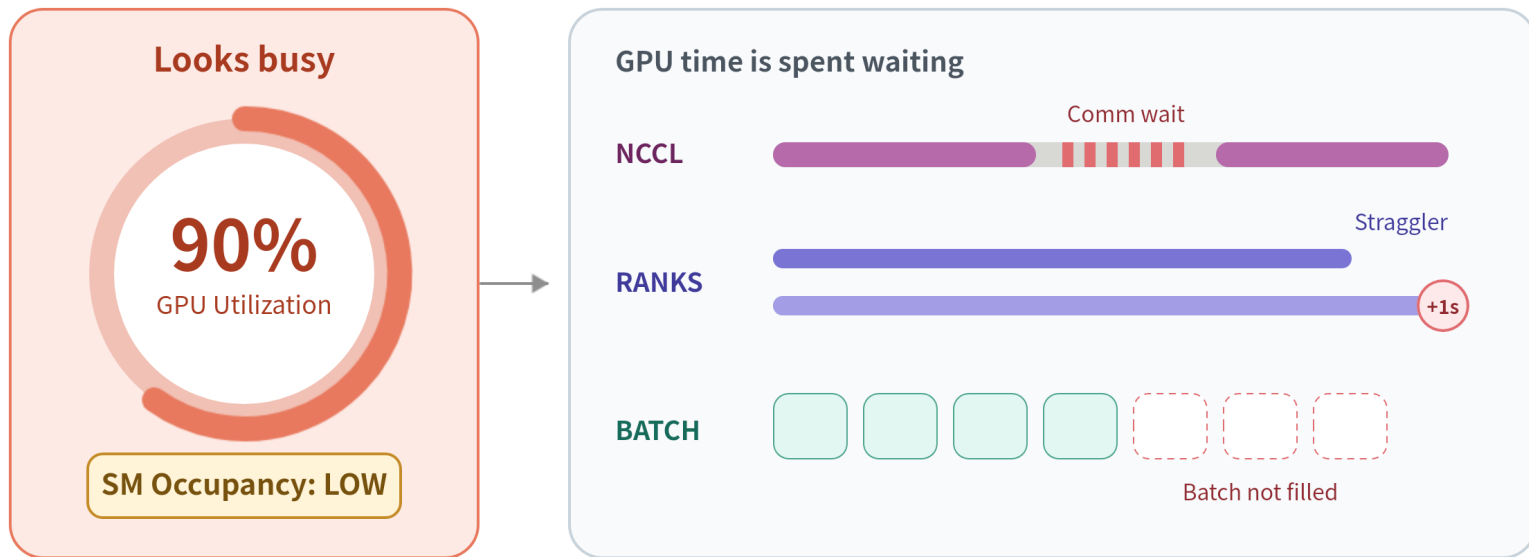
Case 1: Elevated TTFT

Collapses cross-referencing 5–6 panels into one actionable path



Case 2: The “Busy but Idle” GPU

High utilization $\neq$ useful work: find the wait before scaling up

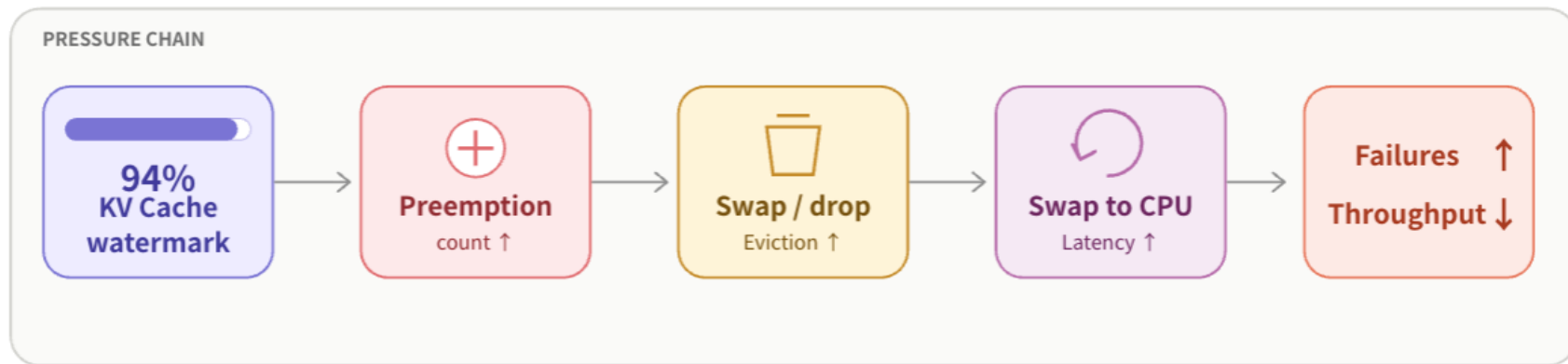


GPU × 2

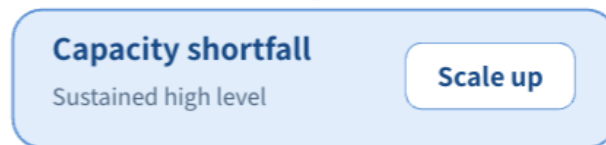
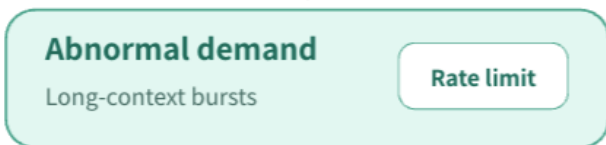
Throughput only +15%

Case 3: Chain Reaction from KV Cache Saturation

Protective preemption avoids a real OOM but pushes pressure into latency, failure rate and throughput



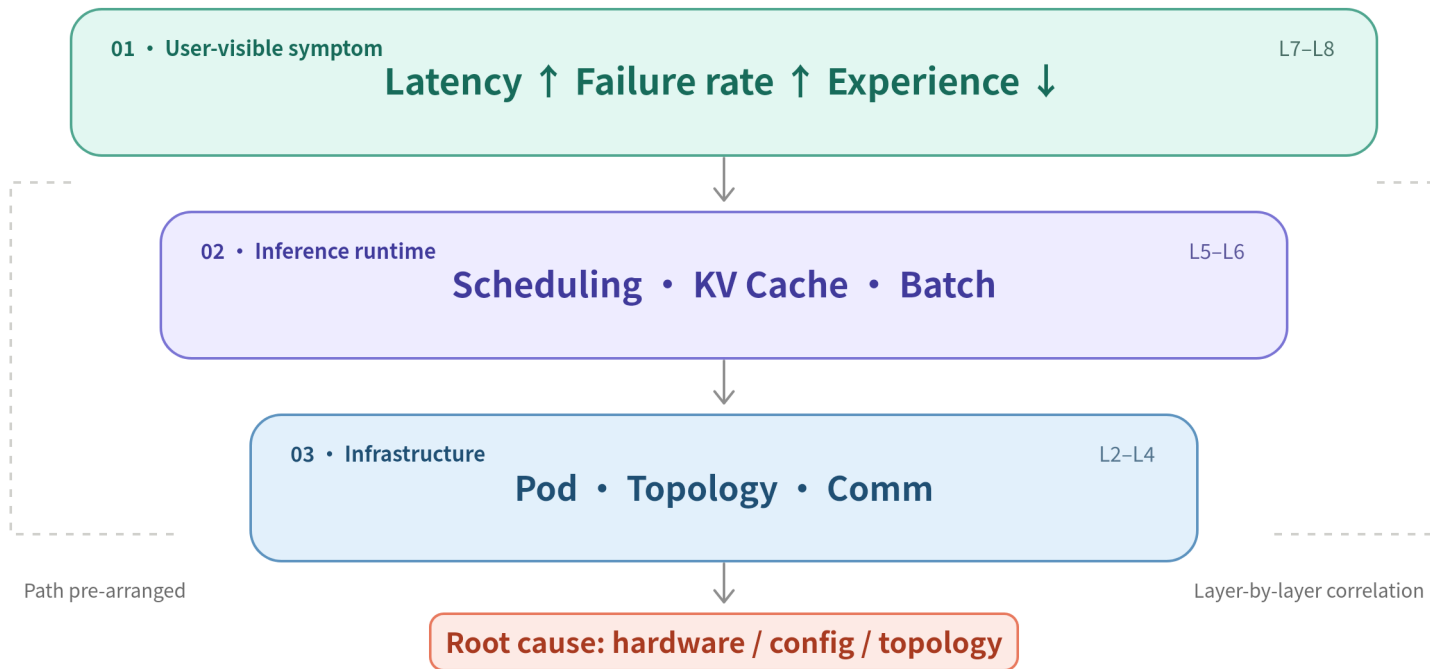
Divert first, then remediate



“Process OOM” is usually just the surface

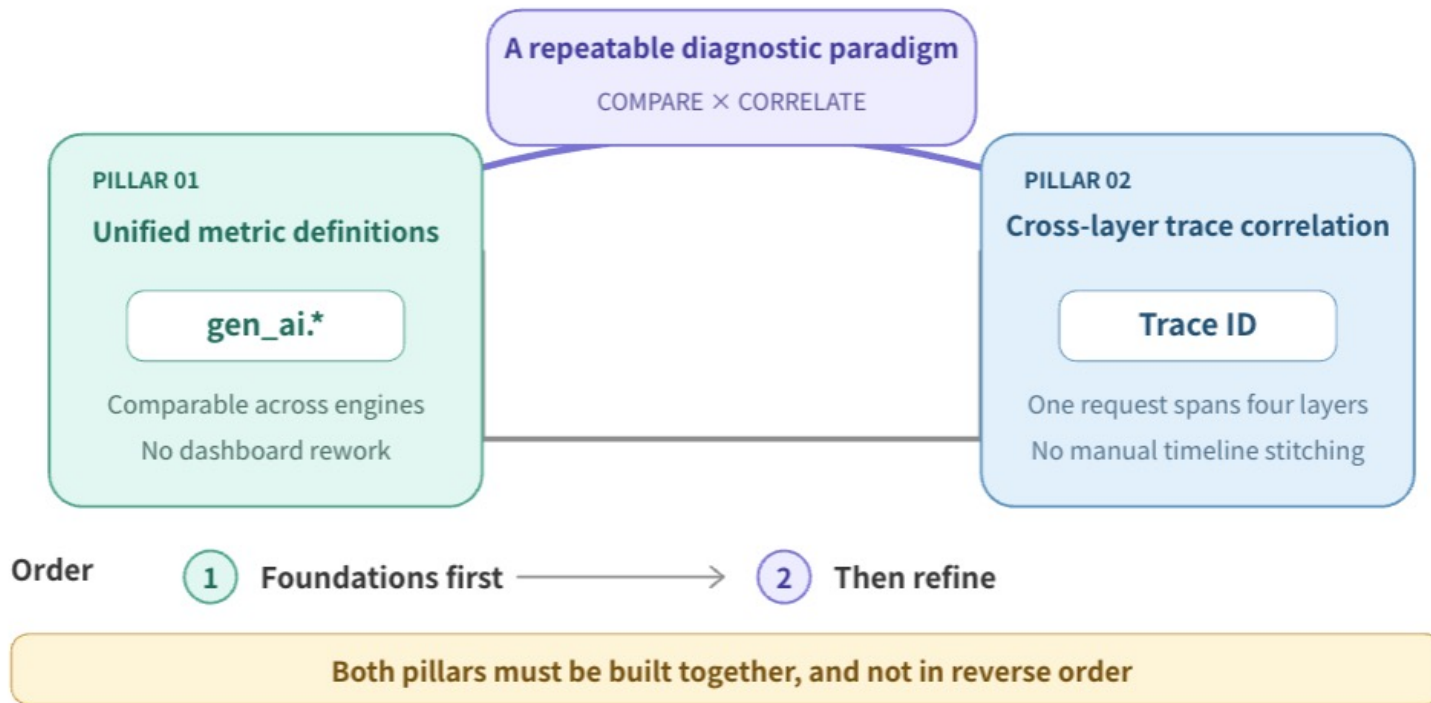
A General Diagnostic Paradigm

Drill 2-3 layers at a time to keep narrowing the search space



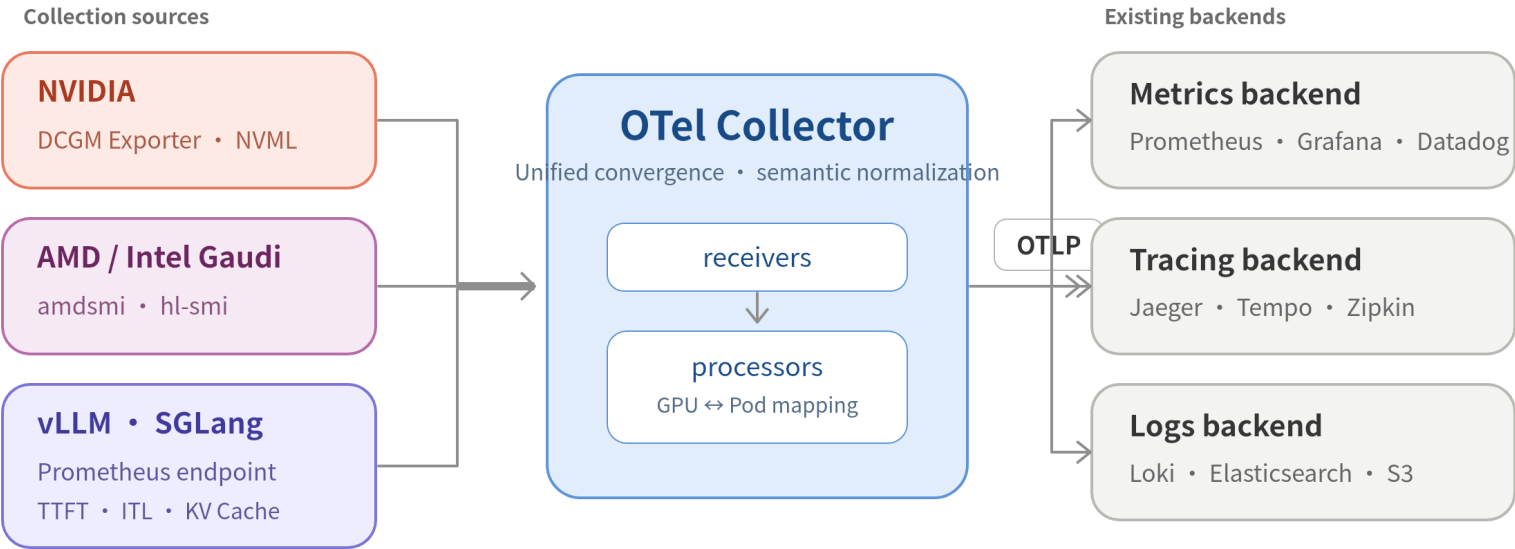
Foundations: Unified Metrics + Trace Correlation

Unified definitions make it comparable; cross-layer traces make it traceable



Open-Source Toolchain Reference

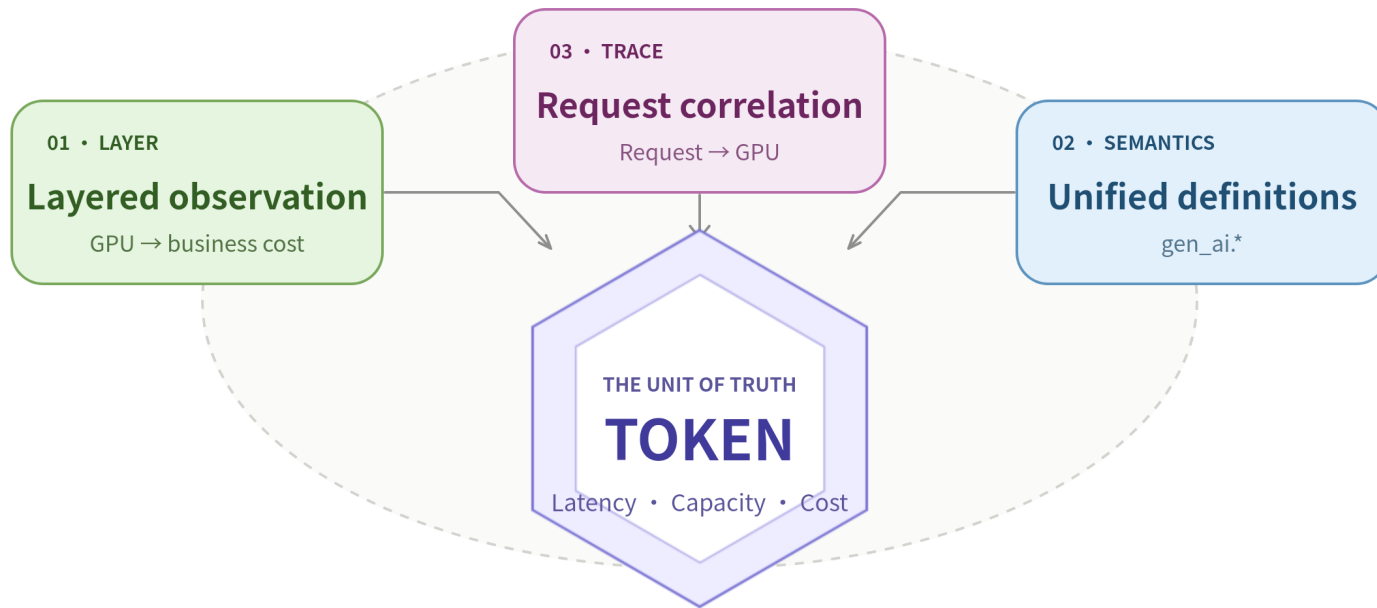
Heterogeneous sources converge in OTEL and feed existing metrics, tracing and logging systems over OTLP



What is unified is collection and transport, not your storage choice

Summary

From GPU utilization to a shared explanation of latency, capacity and cost



Key takeaway GPU utilization is not the finish line; KV Cache and cost per token are the answer



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

Thanks

